Name:- Priyam Das
Roll No:- 2401MC04

OS Lab Assignment 8: Memory Management in xv6

Build and Run Notes

Two ways to boot the kernel were used: `make qemu-nox CPUS=1` for the single-CPU logs of each question, and `make qemu-nox` (2 CPUs) for the full-session log. Inside the xv6 shell I ran `vatest`, `memtest`, `vmprinttest` and `lazytest`. Every terminal capture is saved under `output_logs/`.

Location	What changed
`kernel/`	`vm.c, trap.c, kalloc.c, proc.c, proc.h, exec.c, sysproc.c, syscall.c, syscall.h, defs.h` plus the new header `meminfo.h`
`user/`	`vatest.c, memtest.c, vmprinttest.c, lazytest.c, user.h, usys.S`
`Makefile`	four test programs added to UPROGS, the `-smp N,sockets=N` fix applied, and `-Werror` removed
`output_logs/`	per-question terminal logs and the complete 2-CPU session

Three system calls were added: `va2pa` (22), `getmeminfo` (23) and `vmprint` (24). The work sits on a fresh copy of xv6-public, so nothing from earlier assignments is present; for example, the shared-memory page at `0x60000000` from Assignment 5 does not appear in the page-table dumps. The submission list also mentions cowtest.c, protecttest.c, shmtest.c and pagerepl.c (Q5 to Q8), but the assignment PDF only specifies Q1 to Q4, so only those four questions are implemented here.

Q1. Virtual to Physical Address Translation (va2pa)

Design

To let the system call reuse the page-table walker, `walkpgdir()` in `vm.c` was made non-static and declared in `defs.h`. `sys_va2pa` fetches the address with `argint` and calls `walkpgdir(myproc()->pgdir, va, 0)`. Passing `alloc = 0` guarantees that a lookup never creates a page table as a side effect. The call returns `-1` when the directory entry is absent, when `PTE_P` is clear, or when `PTE_U` is clear. On success it returns `PTE_ADDR(*pte) | (va & 0xFFF)`, and it also prints the directory index, table index, offset and the P/W/U bits through `cprintf` (lines tagged `[kernel]`).

Result (CPUS=1)

Variable	VA	PDX / PTX / offset	PA
global	`0xD78`	0 / 0 / `0xd78`	`0xDEE8D78`
local (stack)	`0x2FCC`	0 / 2 / `0xfcc`	`0xDEE5FCC`
`malloc(100)`	`0xAF98`	0 / 10 / `0xf98`	`0xDF76F98`
`0x7FFFFFFF`	-	511 / 1023 / `0xfff`	-1 (not mapped)
`0x80100000` (kernel)	-	512 / 256 / `0x0`	-1 (PTE_U clear)

After `fork()` the parent and child both print `&global; = 0xD78`, yet the child's physical address is `0xDF2CD78` while the parent's stays `0xDEE8D78`. This is because `copyuvm()` duplicates every frame for the child, so one virtual address maps to two different physical frames.

Q2. Physical Memory Statistics (getmeminfo)

Design and data structures

The `kmem` structure in `kalloc.c` got two new integers, `nfree` and `ntotal`. Inside the `kmem.lock` critical section, `kfree()` increments `nfree` and `kalloc()` decrements it (only when an allocation succeeds). `freerange()` adds to `ntotal` while the kernel boots, and a helper `kmemstat()` returns both numbers under the lock.

```
struct meminfo { int total_pages; int free_pages; int proc_pages; int pt_pages; }; /* meminfo.h
*/
```

`proc_pages` is `sz / PGSIZE`. `pt_pages` comes from a new function `ptpages()` in `vm.c`: one page for the directory plus one for each present PDE. This includes the kernel-half tables that `setupkvm()` gives every process, so the count reflects the true page-table cost (66 for a small program, 64 of those for kernel mappings).

Memory before and after (CPUS=1)

Point in memtest	free_pages	proc_pages	Change
start	56,789	3	-
after `sbrk(16 pages)` + memset	56,773	19	-16
inside the forked child	56,687	19	-86 (16+3 user copy, 66 page tables, 1 kernel stack)
after child exits and is reaped	56,773	19	back to the pre-fork value, so no leak
after `sbrk(-16 pages)`	56,789	3	back to the starting value

Q3. Dumping the Page Table (vmprint)

Design

`vmprint(pgdir, sz)` in `vm.c` loops over all 1024 directory entries. Entries at or above `PDX(KERNBASE)` are only counted. For each other present PDE it prints the index, the PA of its page table and the flags, then goes through that table (`P2V(PTE_ADDR(pde))`) and prints each present PTE with its index, the VA `PGADDR(i, j, 0)`, the frame PA and flags, indented one level deeper. A PTE that has `P` set but `U` clear is labelled `<-- guard page`. The last line compares the number of user pages with `sz/PGSIZE - guard pages`.

Result (vmprinttest)

Call	sz	Entries shown	User pages	Guard	Check
start-up	`0x3000`	pte 0 (code/data), pte 1 (guard, PW-), pte 2 (stack)	2	1	3 - 1 = 2 OK
after `sbrk(8*4096)`	`0xB000`	same as above + pte 3 to 10	10	1	11 - 1 = 10 OK
after `sbrk(-4096)`	`0xA000`	pte 10 is gone	9	1	10 - 1 = 9 OK

Each dump collapses the 64 kernel PDEs into one summary line. The guard page is the one `exec()` blanks with `clearpteu()`, located at VA `0x1000` just under the stack. Since Q4 makes heap pages appear only on first use, `vmprinttest` writes one byte into every new page before calling `vmprint()` so the entries become visible.

Q4. Lazy Heap Allocation

Design and data structures

- **sys_sbrk**: for `n > 0` it simply does `myproc()->sz += n` after rejecting overflow and `sz + n >= KERNBASE`. For `n < 0` it still calls `growproc()`, and therefore `deallocuvm()`, which already ignores non-present PTEs, so only frames that were really allocated get freed.
- **struct proc** gained `uint heapbase`. `exec()` sets it to the top of the user stack, which is where the heap begins, and `fork()` copies it.
- **trap.c, case T_PGFLT** calls `lazyfault()`. It reads `rcr2()` and only accepts the fault if `heapbase <= va < sz` and the PTE is not yet present. It then runs `kalloc()`, `memset(0)` and `mappages(..., PTE_W|PTE_U)` (`mappages` was made non-static), and the faulting instruction is restarted. Every other case prints a reason (beyond sz, below the heap, protection fault, out of memory) and drops into the `default:` handler, which kills a user process or panics for a genuine kernel bug.
- **Kernel touching lazy memory**: `read()` and `write()` use buffers through the user page table, so those faults arrive with `cs` in kernel mode and go through the same `lazyfault()`; the system call just carries on.
- **copyuvm()** in `fork()` now uses `continue` for both a missing page table and a non-present PTE instead of panicking.

Memory before and after (CPUS=1, lazytest)

Step	free_pages	proc_pages	Frames used
before `sbrk(32 MB)`	56,788	4	-
after `sbrk(32 MB)`	56,788	8,196	0
touch 4 pages (0, 8, 16, 32 MB-1)	56,781	8,196	7 (4 data + 3 new page tables)
`read()` / pipe `write()` / `read()` with untouched buffers	-	-	6 (3 data + 3 page tables); syscalls succeed
fork, child touches 1 page, exits, reaped	56,775 to 56,775	-	no leak
child `sbrk(400 MB)` and touches every page	56,775 to 56,775	-	child killed when `kalloc()` fails, no leak
`sbrk(-32 MB)`	56,782	4	data frames freed; the 6 page tables stay until exit (as in stock xv6)

A bad access only kills the child process; the parent and the kernel keep running. The cases tested were an address past sz (`0x2007000`), the stack guard page (`0x2000`, err 7 = protection violation), a kernel address (`0x80100000`) and exhausting memory. Touched pages read back as `0` before any write, confirming they are zero-filled.

Problems Faced and How They Were Fixed

1. **pte_t in defs.h.** Most files include defs.h before mmu.h, so declaring walkpgdir as returning pte_t* broke the build. I declared it with the type uint* instead, which is the same underlying type.
2. **walkpgdir creating tables by accident.** va2pa, vmprint and lazyfault all call it with alloc = 0, so merely inspecting an address never allocates memory.
3. **copyuvm panicking on a lazy heap.** Forking a process with an untouched 32 MB heap hit the panics for "copyuvm: pte should exist" and "page not present". Both checks now skip the entry with continue.
4. **Faults raised inside the kernel.** read() into an untouched buffer faults in kernel mode, which stock xv6 treats as a kernel bug and panics. T_PGFLT is now tested against the process's heap range before the "kernel mode, so panic" path, so these faults are handled invisibly.
5. **Q3 and Q4 interacting.** With lazy allocation, sbrk(8*4096) alone adds no PTEs. vmprinttest therefore touches each new page, so the dump shows them and the sz/PGSIZE - guard identity holds.
6. **Counting page tables.** Each process owns a private copy of the kernel-half page tables (64 of them), so pt_pages and the price of a fork are far bigger than the user part alone. That is why one fork costs 86 frames.
7. **Limitation.** If kalloc() fails while the kernel itself is faulting on a lazy page (out of memory inside a system call), the process is marked killed but the kernel still panics, because the faulting kernel instruction cannot be resumed. User-mode out-of-memory, as tested, is handled cleanly.